

Atividade de Laboratório: Implantação e Avaliação Experimental de Aplicações de Microsserviços com Kubernetes

1. Contexto

Após a etapa inicial da disciplina, dedicada ao estudo e à caracterização de aplicações de microsserviços de código aberto, esta atividade de laboratório tem como objetivo avançar para uma etapa prática de implantação, execução e avaliação experimental dessas aplicações em ambiente containerizado e orquestrado.

A proposta é permitir que os alunos observem, na prática, aspectos importantes do funcionamento de aplicações distribuídas, incluindo implantação em Kubernetes, comportamento sob carga, reação a falhas e uso de escalonamento automático.

Cada equipe deverá selecionar duas aplicações de microsserviços dentre aquelas previamente estudadas no Trabalho 1. As duas aplicações deverão ser implantadas e utilizadas nos experimentos da atividade.

Cada membro da equipe deverá baixar, implantar e executar, em seu próprio computador, os testes solicitados com pelo menos uma das duas aplicações escolhidas pela equipe. O relatório final deverá indicar quais testes foram executados por cada membro, em qual ambiente e com qual aplicação.

2. Objetivo

A atividade tem como objetivo proporcionar uma experiência prática com a implantação e avaliação experimental de aplicações de microsserviços em Kubernetes.

Ao longo do laboratório, espera-se que os alunos sejam capazes de:

- preparar um ambiente local baseado em Kubernetes;
- implantar aplicações de microsserviços previamente analisadas;
- adaptar artefatos de implantação, quando necessário;
- definir cenários funcionais simples para geração de carga;
- conduzir testes de desempenho com diferentes níveis de carga;
- conduzir testes de resiliência com injeção controlada de falhas;
- configurar e observar o comportamento do HPA;
- documentar os procedimentos realizados, os resultados obtidos e as limitações encontradas.

A atividade não tem como objetivo construir um ambiente de produção nem realizar experimentos de grande escala. O foco está na compreensão experimental do comportamento das aplicações em um ambiente local e controlado.

3. Ambiente e Ferramentas

As equipes deverão utilizar um ambiente baseado em Kubernetes, podendo adotar Minikube, Kind, k3d ou alternativa equivalente.

Também deverão utilizar:

- `kubectl`;
- um repositório Git compartilhado;
- uma ferramenta de geração de carga, como Locust ou k6;
- uma ferramenta ou mecanismo para injeção de falhas;
- HPA para os testes de escalonamento automático.

O relatório deverá informar, para cada membro da equipe:

- sistema operacional utilizado;
- processador e quantidade de memória RAM;
- distribuição Kubernetes utilizada;
- aplicação executada localmente;
- testes executados.

4. Aplicações Escolhidas

Cada equipe deverá selecionar duas aplicações de microsserviços dentre aquelas estudadas no Trabalho 1.

Para cada aplicação, o relatório deverá apresentar:

- nome da aplicação;
- repositório principal;
- breve descrição da arquitetura;
- principais serviços da aplicação;
- fluxo funcional escolhido para os testes;
- justificativa da escolha;
- dificuldades de implantação ou adaptação, se houver.

Caso uma aplicação inicialmente escolhida se revele inviável para implantação ou experimentação, a equipe poderá substituí-la por outra aplicação analisada no Trabalho 1. Essa mudança deverá ser registrada e justificada no relatório.

5. Organização Geral da Atividade

A atividade será organizada em três etapas:

- Etapa 1: preparação do ambiente e implantação das aplicações;
- Etapa 2: testes de desempenho;

- Etapa 3: testes de resiliência, testes de escalonamento automático e consolidação final.

O relatório deverá ser atualizado a cada etapa, incorporando os novos procedimentos, resultados e análises. O repositório da equipe também deverá ser atualizado com os artefatos correspondentes.

6. Etapa 1 — Preparação do Ambiente e Implantação das Aplicações

Na primeira etapa, a equipe deverá preparar o ambiente experimental e implantar as duas aplicações escolhidas em Kubernetes.

Para cada aplicação, a equipe deverá:

- identificar o repositório original;
- descrever brevemente a arquitetura;
- identificar o fluxo funcional que será usado nos testes;
- registrar o processo de implantação;
- documentar adaptações realizadas nos manifestos, scripts ou arquivos de configuração;
- demonstrar que a aplicação está funcional em pelo menos um cenário básico de uso.

Entregável da Etapa 1

A equipe deverá entregar uma versão preliminar do relatório contendo:

- identificação da equipe;
- link do repositório do projeto;
- indicação das duas aplicações escolhidas;
- breve descrição da arquitetura de cada aplicação;
- descrição do ambiente utilizado por cada membro;
- relato do processo de implantação de cada aplicação;
- adaptações realizadas nos artefatos originais, se houver;
- evidências de funcionamento das aplicações.

O repositório deverá conter:

- README inicial;
- manifestos Kubernetes utilizados ou adaptados;
- scripts auxiliares, se houver;
- evidências básicas de execução, como capturas de tela, saídas de comandos ou logs relevantes.

7. Etapa 2 — Testes de Desempenho

Na segunda etapa, a equipe deverá realizar testes de desempenho nas duas aplicações escolhidas.

Para cada aplicação, a equipe deverá definir um cenário funcional simples e representativo. Esse cenário será usado pelo script de carga.

Cada membro da equipe deverá executar, em seu próprio computador, os testes de desempenho com pelo menos uma das duas aplicações.

Níveis de Carga

Os testes de desempenho deverão utilizar três níveis de carga:

- carga baixa;
- carga média;
- carga alta.

A carga deverá ser medida pela quantidade de usuários virtuais acessando a aplicação simultaneamente.

A carga alta deverá ser definida experimentalmente. Ela deverá produzir uma taxa de falha não nula, mas não superior a 15%.

A taxa de falha deverá ser calculada como:

taxa de falha = número de requisições com falha / número total de requisições

Se a taxa de falha for zero, a carga provavelmente ainda não atingiu o limite suportado pela aplicação no ambiente local. Nesse caso, a quantidade de usuários virtuais deverá ser aumentada.

Se a taxa de falha for superior a 20%, a carga provavelmente ultrapassou excessivamente o limite suportado pelo ambiente local. Nesse caso, a quantidade de usuários virtuais deverá ser reduzida.

Após definir a carga alta, a equipe deverá escolher:

- uma carga baixa, representando uma situação em que a aplicação opera sem pressão significativa;
- uma carga média, cujos tempos de resposta fiquem em um patamar intermediário entre os tempos observados nas cargas baixa e alta.

O relatório deverá informar e justificar os valores escolhidos para carga baixa, média e alta em cada aplicação.

Métricas Mínimas dos Testes de Desempenho

Para cada teste de desempenho, a equipe deverá registrar pelo menos:

- aplicação testada;
- membro da equipe que executou o teste;
- quantidade de usuários virtuais;
- duração do teste;
- número total de requisições;
- taxa de falha;
- tempo médio de resposta;
- tempo de resposta P95.

A equipe poderá registrar métricas adicionais, mas elas não são obrigatórias.

Entregável da Etapa 2

A equipe deverá entregar uma nova versão do relatório contendo:

- descrição do cenário funcional testado em cada aplicação;
- ferramenta de carga utilizada;
- descrição dos scripts de carga;
- valores definidos para carga baixa, média e alta;
- justificativa para esses valores;
- tabela com os resultados dos testes;
- gráficos simples comparando os resultados das três cargas;
- breve análise dos resultados.

O repositório deverá ser atualizado com:

- scripts de carga;
- arquivos CSV ou equivalentes gerados pela ferramenta de carga;
- gráficos produzidos;
- instruções para reproduzir os testes de desempenho.

8. Etapa 3 — Testes de Resiliência e Escalonamento Automático

Na terceira etapa, a equipe deverá realizar dois tipos de experimento:

- testes de resiliência com injeção de falhas;
- testes de escalonamento automático com HPA.

8.1 Testes de Resiliência

Os testes de resiliência deverão ser realizados nas duas aplicações escolhidas.

Para cada aplicação, a equipe deverá escolher um serviço crítico no qual as falhas serão injetadas.

A escolha do serviço crítico deverá ser justificada no relatório. A justificativa poderá considerar, por exemplo:

- importância do serviço no fluxo funcional testado;
- dependência de outros serviços em relação a ele;
- impacto esperado de sua falha sobre o comportamento da aplicação.

Os testes deverão usar as seguintes taxas de falha injetada:

- 25%;
- 75%.

Essas duas taxas deverão ser utilizadas obrigatoriamente.

Os testes de resiliência deverão utilizar os mesmos níveis de carga definidos nos testes de desempenho:

- carga baixa;
- carga média;
- carga alta.

Para cada aplicação, a equipe deverá executar os testes combinando os três níveis de carga com as duas taxas de falha. Portanto, para cada aplicação, deverão ser executadas nove combinações:

- carga baixa com 25% e 75% de falha;
- carga média com 25% e 75% de falha;
- carga alta com 25% e 75% de falha.

Métricas Mínimas dos Testes de Resiliência

Para cada teste de resiliência, a equipe deverá registrar pelo menos:

- aplicação testada;
- serviço no qual a falha foi injetada;
- membro da equipe que executou o teste;
- nível de carga utilizado;
- taxa de falha injetada;
- taxa de falha observada nas requisições da aplicação;
- tempo médio de resposta;
- tempo de resposta P95.

O relatório deverá deixar clara a diferença entre a taxa de falha injetada no serviço escolhido e a taxa de falha observada nas requisições feitas à aplicação.

8.2 Testes de Escalonamento Automático com HPA

A equipe deverá configurar e testar o HPA em pelo menos um serviço relevante de cada aplicação.

Os testes de escalonamento não deverão utilizar injeção de falhas.

Para os testes de escalonamento, a equipe deverá definir um perfil de carga dinâmico com duração de três minutos:

- no primeiro minuto, a quantidade de usuários virtuais deverá subir gradualmente até atingir um primeiro pico;
- no segundo minuto, a quantidade de usuários virtuais deverá diminuir gradualmente até atingir um determinado piso;
- no terceiro minuto, a quantidade de usuários virtuais deverá subir gradualmente até atingir um segundo pico, maior que o primeiro.

O relatório deverá informar os valores usados para:

- carga inicial;
- primeiro pico;
- piso após a redução;
- segundo pico.

Durante o teste, a equipe deverá observar se o HPA aumentou ou reduziu o número de réplicas do serviço escolhido.

Métricas Mínimas dos Testes de Escalonamento

Para cada teste de escalonamento, a equipe deverá registrar pelo menos:

- aplicação testada;
- serviço configurado com HPA;
- configuração mínima e máxima de réplicas;
- métrica usada pelo HPA;
- perfil de carga utilizado;
- maior número de réplicas observado;
- tempo médio de resposta;
- tempo de resposta P95;
- taxa de falha observada.

8.3 Entregável da Etapa 3

O relatório final deverá consolidar todo o trabalho realizado ao longo das três etapas.

O documento deverá conter:

- identificação da equipe;

- link do repositório;
- descrição das duas aplicações utilizadas;
- descrição do ambiente experimental de cada membro;
- indicação de quais testes foram executados por cada membro;
- descrição do processo de implantação das aplicações;
- descrição dos cenários funcionais testados;
- resultados dos testes de desempenho;
- resultados dos testes de resiliência;
- configuração e resultados dos testes com HPA;
- gráficos e tabelas com os principais resultados;
- dificuldades encontradas;
- limitações dos experimentos;
- lições aprendidas.

O repositório final deverá conter:

- README atualizado;
- manifestos Kubernetes utilizados ou adaptados;
- scripts de carga;
- arquivos de configuração de injeção de falhas;
- arquivos de configuração do HPA;
- arquivos CSV ou equivalentes com os dados brutos;
- gráficos gerados;
- relatório final.

9. Organização Recomendada do Repositório

A equipe deverá manter um repositório Git compartilhado com os artefatos do trabalho.

Uma organização sugerida é:

```

/
├── README.md
├── report/
│   └── relatorio.pdf
├── apps/
│   ├── app1/
│   │   ├── manifests/
│   │   └── scripts/
│   └── app2/

```



```
|   |— manifests/
|   |— scripts/
|— load-tests/
|   |— performance/
|   |— resilience/
|   |— scaling/
|— data/
|   |— raw/
|— figures/
|— hpa/
```

Essa estrutura é apenas uma sugestão. A equipe poderá adotar outra organização, desde que o repositório fique claro e fácil de navegar.

O README deverá explicar:

- quais aplicações foram utilizadas;
- como preparar o ambiente;
- como implantar as aplicações;
- como executar os testes de desempenho;
- como executar os testes de resiliência;
- como executar os testes de escalonamento;
- onde estão os dados brutos;
- onde estão os gráficos.

10. Orientações Gerais

A atividade não deve se limitar à simples execução das aplicações. O foco está em compreender, observar e analisar seu comportamento em um ambiente experimental.

Nem toda aplicação analisada no Trabalho 1 apresentará o mesmo grau de facilidade de implantação ou experimentação. Parte importante do trabalho consiste justamente em lidar com essas dificuldades de forma técnica e bem documentada.

Os resultados obtidos em computadores diferentes podem variar significativamente. Por isso, o relatório deverá descrever o ambiente de cada membro e evitar generalizações indevidas.

Não se espera que os experimentos produzam resultados definitivos sobre as aplicações. Espera-se, porém, que a equipe seja capaz de executar os testes solicitados, coletar dados, apresentar evidências, interpretar os resultados e discutir as limitações observadas.

O trabalho final deverá permitir que outro leitor compreenda o que foi feito, quais artefatos foram utilizados, quais resultados foram obtidos e quais fatores limitaram a análise.